

Recommended Style and Coding Guidelines for Assembly Language

Assembly Language Coding Guidelines

Doc ID: TI-SW-GDLN-004

Version v0.1

Topic of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Notation.....	3
1.3 Implementation Guidelines	3
2. Style Guidelines	4
2.1 General Guidelines	4
2.2 File Preamble.....	4
2.3 Comments.....	5
2.4 Subroutines.....	5
2.5 Control structures	6
2.6 Identifiers & Variables.....	6
2.7 File Naming	7
3. Templates	8
3.1 File Header.....	8
3.2 Subroutine Header	9
4. Summary of Rules.....	10
5. Summary of Guidelines	12
6. Version History	13

1. Introduction

This document describes a general standard for programming in assembly language. The rules and guidelines in this document standardize common code development practices to increase the code readability, productivity and maintainability.

1.1 Purpose

This coding standard is intended to help the developer to:

- ❑ Establish a consistent style
- ❑ Establish an objective basis of code review
- ❑ Guard against common programming errors
- ❑ Limit program complexity

1.2 Notation

The use of work “shall” in the document requires that any project using this document must comply with the stated standard.

1.3 Implementation Guidelines

This document specifies serially numbered set of rules and guidelines for Assembly coding.

The letter ‘R’ prefixed to the number identifies rules and the letter ‘G’ prefixed to the number identifies guidelines.

2. Style Guidelines

2.1 General Guidelines

An assembly language statement consists of four possible fields: a label field, a mnemonic field, an operand field, and a comment.

Labels, mnemonics, operand and comments shall be aligned as follows:

R1. Labels shall start in column 1 and shall be placed on a separate line above the object code referenced by it.

R2. Instruction mnemonics shall start in column 9.

R3. Operands shall start in column 17.

R4. End line comments (comments for lines containing code) shall start in column 30. Comments that describe what a section of code is intended to do shall start in column 3.

```
_dabort
    stmfd r13!, {r0 - r12, lr}; push registers on to stack
    ldr    r12, esmsr3          ; ESM Group3 status register
    ldr    r0,    [r12]
```

R5. No random blank lines must exist in the code. Logical units within a block shall be separated by one blank line. No blank lines shall appear at the end of the file.

R6. Mnemonics, assembler directives shall use lowercase characters.

```
.ref      custom_dabort
.def      _dabort
.asmfunc  _dabort
```

G1 Do not create big source files. Split files into smaller logical files.

2.2 File Preamble

R7. Every assembly file shall have a file header. The file header shall describe purpose of the file. Refer the template shown in [section 3.1](#)

Some items required in the file header are

File Name

Author

Description

Copyright information

- G2 Every source file should have revision history that includes author, date and purpose of the change. Change history shall be kept up to date.

```

; Revision History
;-----
;Version      Date      Author   Change ID      Description
;-----
; ; 00.01.00   6Apr2012   Sunil    0000000000     Initial
Version
;-----

```

- R8. All files shall include the standard comment header with Texas Instruments copyright information.

2.3 Comments

Comments are a very important part of any program even though they are not translated into object code by the assembler. Comments shall be used to document the interface (what is the expected input, what is the expected output, and what it is actually, among others) as it cannot necessarily be derived from the code. Avoid restating implementation in comments, as they tend to get out of sync.

- G3 Avoid unnecessary comments. If something can be said in code as well as comments, say it in code. If something needs to be said and cannot be said in code, say it in comments.

- R9. Code shall not be commented out. If a line of code is not needed it shall be removed. Pieces of code shall not be kept in files for the purpose of history.

- R10. A comment describing what a single line of code is intended to do shall be placed at the end of the line starting at column 30. Comments that describe what a section of code is intended to do shall start in column 3.

```

ldr    r0, ESMSR1_REG ; load the ESMSR1 status register address

```

- G4 Comment density should not be less than 20%

2.4 Subroutines

- R11. Every subroutine shall have a header. This header shall capture its purpose and assumptions, full description of each parameter, the return type, all possible return values, required pre-conditions, and post-conditions. Refer the template shown in [section 3.2](#)

```
-----  
; _coreClearDataFault_  
; Brief description, typically a few lines explaining the  
; purpose of the subroutine  
; Information about routines that are called and any  
; registers that get destroyed shall be mentioned.
```

G5 Subroutine name should identify the action performed or the information provided by it.

G6 Avoid use of multiple returns from subroutines. Multiple returns will reduce the readability of the code. Sub routines should have only one exit point.

2.5 Control structures

R12. Avoid a long series of conditional branch statements in a row followed by sections of code. In other words, avoid *Spaghetti* code.

G7 All branches which exit loops should go to one place.

2.6 Identifiers & Variables

R13. Identifier names that differ only in case shall not be used.

R14. Identifier names shall not contain the underscore character '_' twice in succession.

G8 The name of an identifier should clearly describe its purpose.

G9 Abstract names that are likely to be reused in other parts of the system should be avoided.

G10 Avoid abbreviations in identifier names except where the abbreviation is an industry or project standard.

G11 Variables should be named with nouns or noun phrases. Variables should have close correlation with the names used in the corresponding standard.

R15. All variables are Camel Case starting with a lowercase letter

2.7 File Naming

A system may be composed of files from different sources. Special care shall be given to the naming of the source files because of conflicts with other modules. A standard file naming convention is required to avoid name collisions in a system.

R16. Source files that are distributed, as part of the product shall be uniquely named.

R17. The file name must be Camel Case, where the first letter of an identifier is lowercase and the first letter of each subsequent concatenated word is capitalized.

R18. Filenames shall contain only alphanumeric characters. White spaces, plus, minus and other meta-characters shall be avoided.

R19. File names that differ only in case shall not be used.

3.2 Subroutine Header

```
;*****  
; * @name      Sub Routine name  
; * @brief     Description  
; * Brief description, typically a few lines explaining the  
; * purpose of the subroutine  
; *  
; * Information about routines that are called and any  
; * registers that get destroyed shall be mentioned.  
; *  
; * @param [in]  Input parameters  
; * @param [out] Output parameters  
; *  
; * @return      Return value  
; *  
;*****
```

4. Summary of Rules

- R1. Labels shall start in column 1 and shall be placed on a separate line above the object code referenced by it.
- R2. Instruction mnemonics shall start in column 9.
- R3. Operands shall start in column 17.
- R4. End line comments (comments for lines containing code) shall start in column 30. Comments that describe what a section of code is intended to do shall start in column 3.
- R5. No random blank lines must exist in the code. Logical units within a block shall be separated by one blank line. No blank lines shall appear at the end of the file.
- R6. Mnemonics, assembler directives shall use lowercase characters.
- R7. Every source file shall have a file header. The file header shall describe purpose of the file. Refer the template shown in [section 3.1](#)
- Some items required in the file header are

File Name

Author

Description

Copyright information
- R8. All files shall include the standard comment header with Texas Instruments copyright information.
- R9. Code shall not be commented out. If a line of code is not needed it shall be removed. Pieces of code shall not be kept in files for the purpose of history.
- R10. A comment describing what a single line of code is intended to do shall be placed at the end of the line starting at column 30. Comments that describe what a section of code is intended to do shall start in column 3.
- R11. Every subroutine shall have a header. This header shall capture its purpose and assumptions, full description of each parameter, the return type, all possible return values, required pre-conditions, and post-conditions. Refer the template shown in [section 3.2](#)

- R12. Avoid a long series of conditional branch statements in a row followed by sections of code. In other words, avoid *Spaghetti* code.
- R13. Identifier names that differ only in case shall not be used.
- R14. Identifier names shall not contain the underscore character '_' twice in succession.
- R15. All variables are Camel Case starting with a lowercase letter
- R16. Source files that are distributed, as part of the product shall be uniquely named.
- R17. The file name must be Camel Case, where the first letter of an identifier is lowercase and the first letter of each subsequent concatenated word is capitalized.
- R18. Filenames shall contain only alphanumeric characters. White spaces, plus, minus and other meta-characters shall be avoided.
- R19. File names that differ only in case shall not be used.

5. Summary of Guidelines

- G1 Do not create big source files. Split files into smaller logical files.
- G2 Every source file should have revision history that includes author, date and purpose of the change. Change history shall be kept up to date.
- G3 Avoid unnecessary comments. If something can be said in code as well as comments, say it in code. If something needs to be said and cannot be said in code, say it in comments.
- G4 Comment density should not be less than 20%
- G5 Subroutine name should identify the action performed or the information provided by it.
- G6 Avoid use of multiple returns from subroutines. Multiple returns will reduce the readability of the code. Sub routines should have only one exit point.
- G7 All branches which exit loops should go to one place.
- G8 The name of an identifier should clearly describe its purpose.
- G9 Abstract names that are likely to be reused in other parts of the system should be avoided.
- G10 Avoid abbreviations in identifier names except where the abbreviation is an industry or project standard.
- G11 Variables should be named with nouns or noun phrases. Variables should have close correlation with the names used in the corresponding standard.

6. Version History

S No	Author	Version	Date	Description
1	Siddharth Deshpande	0.1	6-Nov-14	Initial Version